10

Fast polynomial evaluation and interpolation

In the preceding chapters, we have seen extremely fast algorithms for multiplication and division with remainder. We now tackle the next set of problems: evaluation of a polynomial at many points, its inverse problem, namely interpolation, and a substantial generalization, the Chinese Remainder Algorithm.

10.1. Fast multipoint evaluation

We consider the following situation: R is a ring (commutative, with 1, as always), $n \in \mathbb{N}$, $u_0, \ldots, u_{n-1} \in R$, $m_i = x - u_i \in R[x]$, and $m = \prod_{0 \le i < n}(x - u_i)$. Then the evaluation map

$$\begin{array}{rcl} \chi : R[x]/\langle m \rangle & \longrightarrow & R^n \\ f & \longmapsto & (f(u_0), \ldots, f(u_{n-1})) \end{array}$$

is a ring homomorphism. If R is a field, then $R[x]$ and R^n are vector spaces over R, thus R-algebras, and χ is in fact an isomorphism of R-algebras if $u_0, \ldots, u_{n-1}$ are distinct. This is a special case of the Chinese Remainder Theorem 5.3.

In this and the next section, we want to solve the following two problems. We simplify our exposition by assuming that the number n of points is a power of 2. For general n, we have the two options of either adding some "phantom" points or splitting points into two roughly equal halves in the recursive calls. This is discussed after Theorems 8.3 and 9.15.

PROBLEM 10.1. *(Multipoint evaluation) Given $n = 2^k$ for some $k \in \mathbb{N}$, $f \in R[x]$ of degree less than n, and $u_0, \ldots, u_{n-1} \in R$, compute*

$$\chi(f) = (f(u_0), \ldots, f(u_{n-1})).$$

PROBLEM 10.2. *(Interpolation) Given $n = 2^k$ for some $k \in \mathbb{N}$, $u_0, \ldots, u_{n-1} \in R$ such that $u_i - u_j$ is a unit for $i \neq j$, and $v_0, \ldots, v_{n-1} \in R$, compute $f \in R[x]$ of*

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

degree less than n with

$$\chi(f) = (f(u_0), \ldots, f(u_{n-1})) = (v_0, \ldots, v_{n-1}).$$

We have already discussed these problems for a field R and presented algorithms taking time $O(n^2)$ in Chapter 5. The methods of this chapter are *only* of interest in connection with subquadratic multiplication routines, as those from Chapter 8. There we have seen that the evaluation and interpolation problems can be solved with $O(n \log n)$ operations in R if R supports the FFT and $u_i = \omega^i$, where ω is a primitive nth root of unity. Our goal now is a similar bound for the general case.

For arbitrary points $u_0, \ldots, u_{n-1}$, multipoint evaluation can be done with $O(n^2)$ operations in R by using Horner's rule n times. In fact, it can be proved that one evaluation requires at least n multiplications. One might be tempted to think that then n evaluations require at least n^2 multiplications. This is false, and our goal in this section is to see that mass-production of evaluations can be done much cheaper. In the next section, we show the same bound for interpolation.

The idea of the evaluation algorithm is to split the point set $\{u_0, \ldots, u_{n-1}\}$ into two halves of equal cardinality and to proceed recursively with each of the two halves. This leads to a binary tree of depth $\log n$ with root $\{u_0, \ldots, u_{n-1}\}$ and the singletons $\{u_i\}$ for $0 \le i < n$ at the leaves (see Figure 10.1), where log is the binary logarithm.

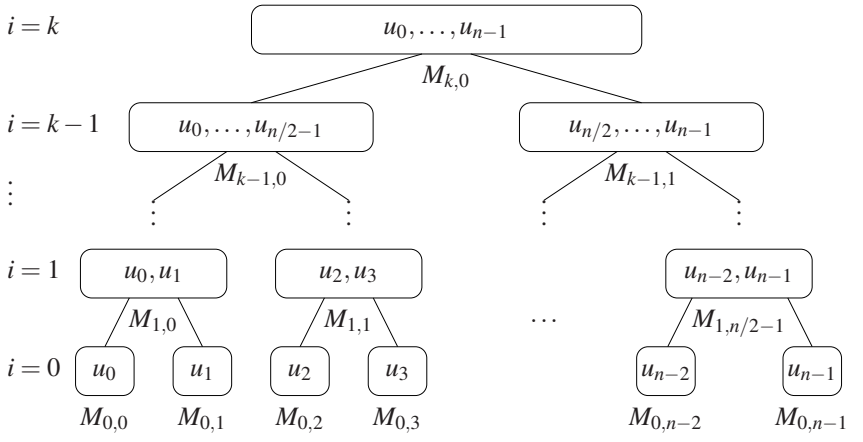


FIGURE 10.1: Subproduct tree for the multipoint evaluation algorithm.

We let $m_i = x - u_i$ as above, and define

$$M_{i,j} = m_{j \cdot 2^i} \cdot m_{j \cdot 2^i + 1} \cdots m_{j \cdot 2^i + (2^i - 1)} = \prod_{0 \le l < 2^i} m_{j \cdot 2^i + l} \tag{1}$$

for $0 \le i \le k = \log n$ and $0 \le j < 2^{k-i}$. Thus each $M_{i,j}$ is a subproduct with 2^i

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

factors of $m = \prod_{0 \le l < n} m_l = M_{k,0}$ and satisfies for each i, j the recursive equations

$$M_{0,j} = m_j, \qquad M_{i+1,j} = M_{i,2j} \cdot M_{i,2j+1}. \tag{2}$$

If R is an integral domain and $u_0, \ldots, u_{n-1}$ are distinct, then $M_{i,j}$ is the monic squarefree polynomial whose zero set is the jth node from the left at level i of the tree in Figure 10.1.

The following algorithm solves the more general problem of computing the subproducts $M_{i,j}$ for *arbitrary* moduli $m_0, \ldots, m_{r-1}$. It proceeds from the leaves to the root of the subproduct tree in Figure 10.1.

ALGORITHM 10.3 Building up the subproduct tree.
Input: $m_0, \ldots, m_{r-1} \in R[x]$, where $r = 2^k$ for some $k \in \mathbb{N}$.
Output: The polynomials $M_{i,j}$ as in (1) for $0 \le i \le k$ and $0 \le j < 2^{k-i}$.

1. **for** $j = 0, \ldots, r-1$ **do** $M_{0,j} \longleftarrow m_j$
2. **for** $i = 1, \ldots, k$ **do**
3. **for** $j = 0, \ldots, 2^{k-i} - 1$ **do** $M_{i,j} \longleftarrow M_{i-1,2j} \cdot M_{i-1,2j+1}$

We recall the multiplication time M (see inside back cover).

LEMMA 10.4. *Algorithm 10.3 correctly computes all subproducts $M_{i,j} \in R[x]$ and takes at most $\mathsf{M}(n) \log r$ operations in R, where $n = \sum_{0 \le i < r} \deg m_i$.*

PROOF. Correctness is clear from (2). Let $d_{i,j} = \deg M_{i,j}$ for all i and j. Step 1 uses no arithmetic operations, and the cost for the ith iteration of step 3 is at most

$$\sum_{0 \le j < 2^{k-i}} \mathsf{M}(d_{i,j}) \le \mathsf{M}\Big(\sum_{0 \le j < 2^{k-i}} d_{i,j} \Big) = \mathsf{M}(n)$$

operations in R, since $\sum_{0 \le j < 2^{k-i}} d_{i,j} = n$. The time estimate follows, since there are $k = \log r$ iterations. □

Exercise 10.8 proves an analogous result for integers. If all m_i have the same degree, then Exercise 10.3 proves the better timing estimate $(\frac{1}{2}\mathsf{M}(n) + O(n)) \log r$. If the degrees of the m_i differ considerably from each other, then the tree in Figure 10.1 is quite unbalanced with respect to the degree. In fact, it is possible to prove a slightly better bound on the arithmetic cost for that case. If $p_0, \ldots, p_{r-1} \in \mathbb{R}$ are positive probabilities that sum to 1, then

$$H(p_0, \ldots, p_{r-1}) = -\sum_{0 \le i < r} p_i \log p_i$$

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

is known from information theory as the **entropy** of $p_0, \ldots, p_{r-1}$. Exercise 10.4 shows that $0 < H(p_0, \ldots, p_{r-1}) \leq \log r$, and $H(p_0, \ldots, p_{r-1}) = \log r$ if and only if $p_0 = \cdots = p_{r-1} = 1/r$. If we organize the subproduct tree in such a way that at each node the total degree of the left and of the right subtree are as well balanced as possible, then the running time bound for computing $m = m_0 \cdots m_{r-1}$ drops to at most $\mathsf{M}(n)(H(\deg m_0/n, \ldots, \deg m_{r-1}/n) + 1)$ (Exercise 10.7); the case where all $\deg m_i$ are equal to n/r yields the bound from Lemma 10.4. The same also applies to the other algorithms in Sections 10.1 through 10.3.

The computation of all the subproducts $M_{i,j}$ can be regarded as a precomputation stage for the fast multipoint evaluation algorithm that we are going to present now. If several polynomials have to be evaluated at the same points $u_0, \ldots, u_{n-1}$, it is sufficient to carry out the precomputation stage only once in advance.

For $n \in \mathbb{N}$, we let $\mathsf{D}(n)$ denote the number of operations in R for dividing a polynomial of degree less than $2n$ by a monic polynomial of degree n in $R[x]$, and assume that $\mathsf{D}(n+m) \geq \mathsf{D}(n) + \mathsf{D}(m)$ and $\mathsf{D}(n) \geq n$ for all $n, m \in \mathbb{N}$. By Theorem 9.6, $\mathsf{D}(n)$ is at most $5\mathsf{M}(n) + O(n)$.

We now present a divide-and-conquer algorithm that, given all subproducts $M_{i,j}$, proceeds top down along the tree in Figure 10.1.

ALGORITHM 10.5 Going down the subproduct tree.
Input: $f \in R[x]$ of degree less than $n = 2^k$ for some $k \in \mathbb{N}$, $u_0, \ldots, u_{n-1} \in R$, and the subproducts $M_{i,j}$ from (1).
Output: $f(u_0), \ldots, f(u_{n-1}) \in R$.

1. **if** $n = 1$ **then return** f
2. $r_0 \longleftarrow f \text{ rem } M_{k-1,0}, \quad r_1 \longleftarrow f \text{ rem } M_{k-1,1}$
3. **call** the algorithm recursively to compute $r_0(u_0), \ldots, r_0(u_{n/2-1})$
4. **call** the algorithm recursively to compute $r_1(u_{n/2}), \ldots, r_1(u_{n-1})$
5. **return** $r_0(u_0), \ldots, r_0(u_{n/2-1}), r_1(u_{n/2}), \ldots, r_1(u_{n-1})$

THEOREM 10.6.
Algorithm 10.5 works correctly and takes at most $\mathsf{D}(n)\log n$ *operations in* R, *which is at most* $(5\mathsf{M}(n) + O(n))\log n$ *or* $O(\mathsf{M}(n)\log n)$.

PROOF. We prove the correctness by induction on k. If $k = 0$, then f is constant, and the algorithm outputs the correct value in step 1. Otherwise, if $k \geq 1$, we may inductively assume that the results of steps 3 and 4 are correct. Let $q_0 =$

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

f quo $M_{k-1,0}$ and $q_1 = f$ quo $M_{k-1,1}$. Then

$$f(u_i) = \begin{cases} q_0(u_i)M_{k-1,0}(u_i) + r_0(u_i) = r_0(u_i) & \text{if } 0 \le i < \dfrac{n}{2}, \\ q_1(u_i)M_{k-1,1}(u_i) + r_1(u_i) = r_1(u_i) & \text{if } \dfrac{n}{2} \le i < n. \end{cases}$$

Let $T(n) = T(2^k)$ denote the cost for the recursive process. Then $T(1) = 0$ and

$$T(2^k) = 2T(2^{k-1}) + 2\mathsf{D}(2^{k-1})$$

for $k \ge 1$, so that $T(2^k) \le 2k \cdot \mathsf{D}(2^{k-1}) \le \mathsf{D}(n)\log n$, by Lemma 8.2, and the claim follows from Theorem 9.6. □

Putting things together, we obtain the following algorithm for fast multipoint evaluation.

ALGORITHM 10.7 Fast multipoint evaluation.
Input: $f \in R[x]$ of degree less than $n = 2^k$ for some $k \in \mathbb{N}$ and $u_0, \ldots, u_{n-1} \in R$.
Output: $f(u_0), \ldots, f(u_{n-1}) \in R$.

1. **call** Algorithm 10.3 with input $(x - u_0), \ldots, (x - u_{n-1})$ to compute the subproducts $M_{i,j}$ as in (1)
2. **call** Algorithm 10.5 with input f, the points u_i, and the subproducts $M_{i,j}$
   return its results

COROLLARY 10.8.
Evaluation of a polynomial in $R[x]$ of degree less than n at n points in R can be performed using at most $(\frac{11}{2}\mathsf{M}(n) + O(n))\log n$ or $O(\mathsf{M}(n)\log n)$ operations in R.

The time bound follows from Exercise 10.3 and Theorem 10.6. Exercise 10.9 proves the smaller bound $(1 + \frac{7}{2}\log n)(\mathsf{M}(n) + O(n))$. Exercise 10.11 shows that if many evaluations at the same set of points have to be performed, then all data depending only on the evaluation points may be precomputed and stored, and the cost drops to essentially $(2\mathsf{M}(n) + O(n))\log n$.

10.2. Fast interpolation

We recall the Lagrange interpolation formula from Chapter 5. Given distinct $u_0, \ldots, u_{n-1}$ in a field F and arbitrary $v_0, \ldots, v_{n-1} \in F$, the unique polynomial

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

$f \in F[x]$ of degree less than n that takes the value v_i at the point u_i for all i is $f = \sum_{0 \le i < n} v_i s_i m/(x-u_i)$, where $m = (x-u_0)\cdots(x-u_{n-1})$, as before, and

$$s_i = \prod_{j \ne i} \frac{1}{u_i - u_j}. \tag{3}$$

Over a ring R, this is still valid if we demand that $u_i - u_j$ is a unit for $i \ne j$. Theorem 10.13 below shows that this condition is also necessary in the general case.

We first explain an idea to compute the s_i fast. The formal derivative m is $m' = \sum_{0 \le j < n} m/(x-u_j)$, and since $m/(x-u_i)$ vanishes at all points u_j with $j \ne i$, we have

$$m'(u_i) = \left.\frac{m}{x-u_i}\right|_{x=u_i} = \frac{1}{s_i}. \tag{4}$$

Given m, the computation of all the s_i amounts to one evaluation of m' at n points, at a cost of $O(\mathsf{M}(n)\log n)$ operations in R, plus n inversions.

The following divide-and-conquer algorithm is the core of the fast interpolation algorithm. It proceeds from the leaves to the root of the tree in Figure 10.1.

ALGORITHM 10.9 Linear combination for linear moduli.

Input: $u_0, \ldots, u_{n-1}, c_0, \ldots, c_{n-1} \in R$, where $n = 2^k$ for some $k \in \mathbb{N}$, and the polynomials $M_{i,j}$ as in (1).

Output: $\displaystyle\sum_{0 \le i < n} c_i \frac{m}{x-u_i} \in R[x]$, where $m = (x-u_0)\cdots(x-u_{n-1})$.

1. **if** $n = 1$ **then return** c_0
2. **call** the algorithm recursively to compute $r_0 = \displaystyle\sum_{0 \le i < n/2} c_i \frac{M_{k-1,0}}{x-u_i}$
3. **call** the algorithm recursively to compute $r_1 = \displaystyle\sum_{n/2 \le i < n} c_i \frac{M_{k-1,1}}{x-u_i}$
4. **return** $M_{k-1,1} r_0 + M_{k-1,0} r_1$

THEOREM 10.10.

Algorithm 10.9 takes at most $(\mathsf{M}(n) + O(n))\log n$ *or* $O(\mathsf{M}(n)\log n)$ *arithmetic operations in* R *to correctly compute the result.*

PROOF. As usual, we prove the correctness by induction on k. If $k = 0$, then $m = x - u_0$, and the output in step 1 is correct. If $k \ge 1$, then the results of the recursive calls in steps 2 and 3 are correct by the induction hypothesis, and the algorithm outputs the correct result in step 4 since $m = M_{k-1,0} \cdot M_{k-1,1}$.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

Let $T(n) = T(2^k)$ denote the cost of the algorithm. The cost for the individual steps is 0 for step 1, $T(n/2)$ for each of the steps 2 and 3, and at most $2\mathsf{M}(n/2+1)+n \in \mathsf{M}(n)+O(n)$ (Exercise 8.34) for step 4. (The "+1" comes from our convention that $\mathsf{M}(n)$ is the time to multiply polynomials of degree less than n.) Thus $T(1) = 0$ and $T(n) \leq 2T(n/2) + \mathsf{M}(n) + cn$ for $n > 1$ and some constant $c \in \mathbb{R}$, and Lemma 8.2 yields the claim. □

Putting things together, we have the following fast interpolation algorithm.

ALGORITHM 10.11 Fast interpolation.

Input: $u_0, \ldots, u_{n-1} \in R$ such that $u_i - u_j$ is a unit for $i \neq j$, and $v_0, \ldots, v_{n-1} \in R$, where $n = 2^k$ for some $k \in \mathbb{N}$.

Output: The unique polynomial $f \in R[x]$ of degree less than n such that $f(u_i) = v_i$ for $0 \leq i < n$.

1. **call** Algorithm 10.3 with input $m_0 = x - u_0, \ldots, m_{n-1} = x - u_{n-1}$ to compute the polynomials $M_{i,j}$ as in (1)
2. $m \longleftarrow M_{k,0}$
   call Algorithm 10.5 with input $f = m'$, $u_0, \ldots, u_{n-1}$, and the $M_{i,j}$ to evaluate m' at $u_0, \ldots, u_{n-1}$
   for $i = 0, \ldots, n-1$ **do** $s_i \longleftarrow \dfrac{1}{m'(u_i)}$
3. **call** Algorithm 10.9 with input $u_0, \ldots, u_{n-1}$, $v_0 s_0, \ldots, v_{n-1} s_{n-1}$, and the $M_{i,j}$
   return its result

COROLLARY 10.12.

Algorithm 10.11 solves the interpolation problem 10.2 over a (commutative) ring R using at most $(\frac{13}{2}\mathsf{M}(n)+O(n))\log n$ or $O(\mathsf{M}(n)\log n)$ operations in R.

PROOF. The cost for step 1 is at most $(\frac{1}{2}\mathsf{M}(n)+O(n))\log n$, by Exercise 10.3. The cost for step 2 is at most $(5\mathsf{M}(n)+O(n))\log n$ operations, by Theorem 10.6, including the computation of m' and the final modular inversions. Finally, step 3 takes no more than $(\mathsf{M}(n)+O(n))\log n$ operations, by Theorem 10.10. □

Exercise 10.11 shows that if many interpolations at the same set of points have to be performed, then all data depending only on the interpolation points may be precomputed and stored, and the cost drops to essentially $(\mathsf{M}(n)+O(n))\log n$.

10.3. Fast Chinese remaindering

The ideas and algorithms of the previous sections carry over to monic moduli of arbitrary degree in $R[x]$ and to $\mathbb{Z}$. We only discuss the polynomial case in detail and refer to the exercises for the integer analog.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

Let R be a field, $m_0,\dots,m_{r-1}\in R[x]$ nonconstant and pairwise coprime, $m = m_0\cdots m_{r-1}$, and $n = \deg m$. Then evaluation and interpolation correspond to computing the Chinese Remainder isomorphism

$$\begin{array}{rcl} \chi: R[x]/\langle m\rangle & \longrightarrow & R[x]/\langle m_0\rangle \times \ldots \times R[x]/\langle m_{r-1}\rangle, \\ f \bmod m & \longmapsto & (f \bmod m_0,\dots,f \bmod m_{r-1}), \end{array} \tag{5}$$

and its inverse. For arbitrary coefficient rings, we have the following variant of the Chinese Remainder Theorem 5.3; Exercise 10.13 asks for a proof.

THEOREM 10.13.
Let $r \geq 1$, R be a ring (commutative, with 1, as always), $m_0,\dots,m_{r-1}\in R[x]$ monic and nonconstant, and $m = m_0\cdots m_{r-1}$. Then the following are equivalent.

(i) The ring homomorphism χ in (5) is an isomorphism.

(ii) There exist polynomials $s_0,\dots,s_{r-1}\in R[x]$ such that $\sum_{0\leq i<r} s_i m/m_i = 1$.

(iii) For $i\neq j$ there exist polynomials $s_{ij},t_{ij}\in R[x]$ such that $s_{ij}m_j + t_{ij}m_i = 1$.

(iv) $\operatorname{res}(m_i,m_j)\in R^\times$ for $i\neq j$.

If each $m_i = x - u_i$ is linear, with $u_i\in R$, then

$$\operatorname{res}(m_i,m_j) = \det\begin{pmatrix} 1 & 1 \\ -u_i & -u_j \end{pmatrix} = u_i - u_j,$$

and (iv) is equivalent to $u_i - u_j\in R^\times$ for $i\neq j$.

Let us assume for simplicity that $r = 2^k$ is a power of 2 for some $k\in\mathbb{N}$. For the moment, we also drop the requirement that $\operatorname{res}(m_i,m_j)\in R^\times$ for $i\neq j$. We build the subproduct tree with $M_{i,j}\in F[x]$ as in (1). The following algorithm generalizes Algorithm 10.5.

ALGORITHM 10.14 Fast simultaneous reduction with precomputation.
Input: Monic nonconstant moduli $m_0,\dots,m_{r-1}\in R[x]$, where $r = 2^k$ with $k\in\mathbb{N}$, $f\in R[x]$ of degree less than $n = \sum_{0\leq i<r}\deg m_i$, and the polynomials $M_{i,j}$ from (1).
Output: $f \operatorname{rem} m_0,\dots,f \operatorname{rem} m_{r-1}\in R[x]$.

1. **if** $r = 1$ **then return** f
2. $f_0 \longleftarrow f \operatorname{rem} M_{k-1,0}$, $\quad f_1 \longleftarrow f \operatorname{rem} M_{k-1,1}$
3. **call** the algorithm recursively to compute $f_0 \operatorname{rem} m_0,\dots,f_0 \operatorname{rem} m_{r/2-1}$
4. **call** the algorithm recursively to compute $f_1 \operatorname{rem} m_{r/2},\dots,f_1 \operatorname{rem} m_{r-1}$
5. **return** $f_0 \operatorname{rem} m_0,\dots,f_0 \operatorname{rem} m_{r/2-1}, f_1 \operatorname{rem} m_{r/2},\dots,f_1 \operatorname{rem} m_{r-1}$

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

THEOREM 10.15.
Algorithm 10.14 works correctly and takes no more than $(10\mathsf{M}(n)+O(n))\log r$ *or* $O(\mathsf{M}(n)\log r)$ *operations in* R.

PROOF. The correctness proof is similar to that of Theorem 10.6 and left as Exercise 10.15. For the cost analysis, we see that the algorithm works from the root to the leaves along the binary tree formed by the subproducts $M_{i,j}$. The cost for a vertex $M_{i,j}$ with $i \geq 1$ is the cost for dividing a polynomial of smaller degree than $\deg M_{i,j}$ by $M_{i-1,2j}$ and $M_{i-1,2j+1}$ with remainder, using at most $2\mathsf{D}(\deg M_{i,j})$ ring operations. The total cost at level i is then at most $2\sum_{0\leq j<2^i}\mathsf{D}(\deg M_{i,j}) \leq 2\mathsf{D}(n)$, as in the proof of Lemma 10.4, and the claim follows from Theorem 9.6 and the fact that there are $\log r$ levels. □

ALGORITHM 10.16 Fast simultaneous modular reduction.
Input: Monic nonconstant moduli $m_0,\ldots,m_{r-1} \in R[x]$, where $r = 2^k$ with $k \in \mathbb{N}$, and $f \in R[x]$ of degree less than $n = \sum_{0\leq i<r}\deg m_i$.
Output: $f \text{ rem } m_0,\ldots,f \text{ rem } m_{r-1} \in R[x]$.

1. **call** Algorithm 10.3 with input $m_0,\ldots,m_{r-1}$ to compute the subproducts $M_{i,j}$ as in (1)
2. **call** Algorithm 10.14 with input $m_0,\ldots,m_{r-1}$, f, and the subproducts $M_{i,j}$ **return** its results

COROLLARY 10.17.
Given monic nonconstant polynomials $m_0,\ldots,m_{r-1} \in R[x]$, *where* $r \in \mathbb{N}$ *is a power of* 2, *and* $f \in R[x]$ *of degree less than* $n = \sum_{0\leq i<r}\deg m_i$, *Algorithm 10.16 computes* $f \text{ rem } m_0,\ldots,f \text{ rem } m_{r-1}$ *using at most* $(11\mathsf{M}(n)+O(n))\log r$ *or* $O(\mathsf{M}(n)\log r)$ *operations in* R.

Exercise 10.17 gives a better analysis of Algorithm 10.16 when all moduli have the same degree.

For the fast Chinese Remainder Algorithm, we recall the generalization of Lagrange's formula from Chapter 5. Given pairwise distinct and nonconstant moduli $m_0,\ldots,m_{r-1} \in F[x]$ for a field F and polynomials $v_0,\ldots,v_{r-1} \in F[x]$ with $\deg v_i < \deg m_i$ for all i, there is a unique polynomial $f \in F[x]$ of degree less than $n = \sum_{0\leq i<r}\deg m_i$ satisfying $f \equiv v_i \bmod m_i$ for all i, and it is given by $f = \sum_{0\leq i<r}(v_i s_i \text{ rem } m_i)m/m_i$, where $m = m_0\cdots m_{r-1}$ and $s_i \in F[x]$ is an inverse of m/m_i modulo m_i. Theorem 10.13 implies that this is true for arbitrary coefficient rings R if we require $\text{res}(m_i,m_j) \in R^\times$ for $i \neq j$.

As in the case of interpolation, we first address the task of computing the s_i. This need be done only once if several computations with the same set of moduli are to be executed.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

ALGORITHM 10.18 Simultaneous inverse computation.
Input: $m_0, \ldots, m_{r-1} \in R[x]$ monic and nonconstant such that $\operatorname{res}(m_i, m_j) \in R^\times$ for $i \neq j$, where $r = 2^k$ for some $k \in \mathbb{N}$, and $m = m_0 \cdots m_{r-1}$.
Output: $s_0, \ldots, s_{r-1} \in R[x]$ with $s_i \dfrac{m}{m_i} \equiv 1 \bmod m_i$ and $\deg s_i < \deg m_i$ for all i.

1. **call** Algorithm 10.16 to compute $m \operatorname{rem} m_i^2$ for all i
2. **for** $i = 0, \ldots, r-1$ compute $\dfrac{m}{m_i} \operatorname{rem} m_i$
3. **for** $i = 0, \ldots, r-1$ compute $s_i \in R[x]$ with $s_i \cdot \left(\dfrac{m}{m_i} \operatorname{rem} m_i \right) \equiv 1 \bmod m_i$ and $\deg s_i < \deg m_i$, using the Extended Euclidean Algorithm if R is a field and Exercise 6.15 otherwise
4. **return** $s_0, \ldots, s_{r-1}$

LEMMA 10.19. *Algorithm 10.18 works correctly. If R is a field, then it takes $O(\mathsf{M}(n) \log n)$ operations in R for $n = \deg m$.*

PROOF. Let R be a field and $d_i = \deg m_i$ for $0 \leq i < r$. The cost for step 1 is $O(\mathsf{M}(2n) \log r)$ ring operations, including the cost for computing all m_i^2. Step 2 costs $\mathsf{D}(d_i) \in O(\mathsf{M}(d_i))$ for m_i. In Chapter 11, we will see that step 3 can be done with $O(\mathsf{M}(d_i) \log d_i)$ operations in R for each i. Using

$$\sum_{0 \leq i < r} \mathsf{M}(d_i) \leq \mathsf{M}\Big(\sum_{0 \leq i < r} d_i \Big) = \mathsf{M}(n),$$

we have a cost of $O(\mathsf{M}(n))$ and $O(\mathsf{M}(n) \log n)$ for steps 2 and 3, respectively, and the claim follows. □

Here is the corresponding generalization of Algorithm 10.9.

ALGORITHM 10.20 Linear combination.
Input: $m_0, \ldots, m_{r-1} \in R[x]$ monic and nonconstant, where $r = 2^k$ for some $k \in \mathbb{N}$, $c_0, \ldots, c_{r-1} \in R[x]$ with $\deg c_i < \deg m_i$ for all i, and the polynomials $M_{i,j}$ as in (1).
Output: The polynomial $f = \displaystyle\sum_{0 \leq i < r} c_i \frac{m}{m_i} \in R[x]$, where $m = m_0 \cdots m_{r-1}$.

1. **if** $r = 1$ **then return** c_0
2. **call** the algorithm recursively to compute $r_0 = \displaystyle\sum_{0 \leq i < r/2} c_i \frac{M_{k-1,0}}{m_i}$

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

3. **call** the algorithm recursively to compute $r_1 = \sum_{r/2 \le i < r} c_i \frac{M_{k-1,1}}{m_i}$
4. **return** $M_{k-1,1} r_0 + M_{k-1,0} r_1$

THEOREM 10.21.
Algorithm 10.20 works correctly. If $\sum_{0 \le i < r} \deg m_i < n$, then it takes no more than $(2\mathsf{M}(n) + O(n)) \log r$ or $O(\mathsf{M}(n) \log r)$ arithmetic operations in R.

The correctness proof is analogous to the proof of Theorem 10.10 and the running time bound can be obtained by considering the same binary tree as in the proof of Theorem 10.15. The details can be found in Exercise 10.16.

ALGORITHM 10.22 Fast Chinese Remainder Algorithm.
Input: $m_0, \ldots, m_{r-1} \in R[x]$ such that $\operatorname{res}(m_i, m_j) \in R^\times$ for $i \neq j$, where R is a ring (commutative, with 1), $r = 2^k$ for some $k \in \mathbb{N}$, and $v_0, \ldots, v_{r-1} \in R[x]$ with $\deg v_i < \deg m_i$ for all i.
Output: The unique polynomial $f \in R[x]$ of degree less than $n = \sum_{0 \le i < r} \deg m_i$ such that $f \equiv v_i \bmod m_i$ for $0 \le i < n$.

1. **call** Algorithm 10.3 with input $m_0, \ldots, m_{r-1}$ to compute the polynomials $M_{i,j}$ as in (1)
2. **call** Algorithm 10.18 with input $m_0, \ldots, m_{r-1}$ and $m = M_{k,0}$ to compute polynomials $s_i \in R[x]$ with $s_i \frac{m}{m_i} \equiv 1 \bmod m_i$ and $\deg s_i < \deg m_i$ for all i
3. **call** Algorithm 10.20 with input $m_0, \ldots, m_{r-1}$, $v_0 s_0 \operatorname{rem} m_0, \ldots, v_{r-1} s_{r-1} \operatorname{rem} m_{r-1}$, and the polynomials $M_{i,j}$
   return its result

COROLLARY 10.23.
Given $m_0, \ldots, m_{r-1} \in F[x]$ monic and pairwise coprime, where F is a field, and $v_0, \ldots, v_{r-1} \in F[x]$ with $\deg v_i < \deg m_i$ for all i, we can compute the unique solution $f \in F[x]$ of degree less than $n = \sum_{0 \le i < r} \deg m_i$ of the Chinese Remainder Problem

$$f \equiv v_i \bmod m_i \text{ for } 0 \le i < r$$

using $O(\mathsf{M}(n) \log n)$ operations in F.

Exercise 10.17 gives an explicit constant for the leading cost term of Algorithm 10.22 when all moduli have the same degree.

We only state the corresponding results for the integer case. Algorithms 10.14 and 10.20 carry over almost literally; the details are left as an exercise.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

THEOREM 10.24.
Given $m_0, \ldots, m_{r-1} \in \mathbb{N}_{\geq 2}$ and $f \in \mathbb{N}$ less than $m = \prod_{0 \leq i < r} m_i$, we can compute $f \bmod m_0, \ldots, f \bmod m_{r-1}$ using $O(\mathsf{M}(\log m) \log r)$ word operations.

THEOREM 10.25.
Given pairwise coprime integers $m_0, \ldots, m_{r-1} \in \mathbb{N}_{\geq 2}$ and $v_0, \ldots, v_{r-1} \in \mathbb{N}$ such that $v_i < m_i$ for all i, we can compute the unique solution $f \in \mathbb{N}$ with $f < m = \prod_{0 \leq i < r} m_i$ of the Chinese Remainder Problem $f \equiv v_i \bmod m_i$ for $0 \leq i < r$, at a cost of $O(\mathsf{M}(\log m) \operatorname{loglog} m)$ word operations.

Notes. Pan (1966) proved the optimality of Horner's rule. The results in Sections 10.1 through 10.3 are based on Lipson (1971), Fiduccia (1972a), Horowitz (1972), Moenck & Borodin (1972), and Borodin & Moenck (1974). Borodin & Munro (1975) give a comprehensive treatment.

Exercises.

10.1 Let $f = 8x^7 + 7x^6 + 6x^5 + 5x^4 + 4x^3 + 3x^2 + 2x + 1 \in \mathbb{Q}[x]$. Trace Algorithm 10.7 to evaluate f at the eight integer points $-3, -2, \cdots, 4$. In the recursive algorithm 10.5, you need only execute the last recursive step and may compute its inputs directly.

10.2 Let R be a ring (commutative, with 1), $n \in \mathbb{N}$ a power of 2, and $k \in \mathbb{N}$. Show that you can evaluate a polynomial of degree less than kn at n points from R using $(2k + 1 + \frac{11}{2} \log n)\mathsf{M}(n) + O((k + \log n)n)$ additions and multiplications in R. Hint: Exercise 9.16.

10.3 Let R be a ring (commutative, with 1), $m_0, \ldots, m_{r-1} \in R[x]$ of degree $d > 0$, and $n = rd$, where r is a power of two. Using Exercise 8.34, prove that Algorithm 10.3 takes only $(\mathsf{M}(n/2) + O(n)) \log r$ or $(\frac{1}{2}\mathsf{M}(n) + O(n)) \log r$ ring operations.

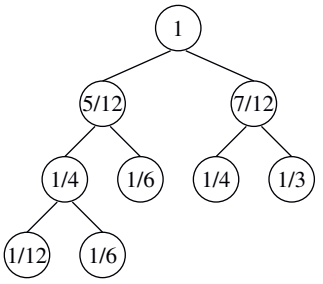
10.4* Let $n \in \mathbb{N}$ and $p_1, \ldots, p_n \in \mathbb{R}_{>0}$ such that $\sum_{1 \leq i \leq n} p_i = 1$.

(i) Prove that $H(p_1, \ldots, p_n) \geq 0$, with equality if and only if $n = 1$.

(ii) Prove that $H(p_1, \ldots, p_n) \leq \log n$, with equality if and only if $p_1 = \cdots = p_n = 1/n$. Hint: Use $\ln x \leq x - 1$ for all positive $x \in \mathbb{R}$, with equality if and only if $x = 1$, and apply this to the expression $\sum_{1 \leq i \leq n} p_i \ln(1/p_i n)/\ln 2$. (Recall that $\log = \log_2$ and $\ln = \log_e$.)

10.5** A **mobile** is a complete binary tree with additive node weights, so that each node has either no child (it is a **leaf**) or two children (it is an **internal node**), and a positive real number, its **weight**. The weight of an internal node is the sum of the weights of its two children. By induction, the weight of any node in a mobile is the sum of the weights of all leaves in the subtree rooted at that node, and in particular, the weight of the root is the sum of the weights of all leaves. A **stochastic mobile** is a mobile with root weight 1; it is useful to imagine all weights to be probabilities. Any mobile can be turned into a stochastic mobile by dividing all weights by the root weight. The **depth** of a node in a mobile is the length of the path from the root to that node. The **average depth** of a stochastic mobile with n leaves is $\sum_{1 \leq i \leq n} p_i d_i$, where p_i is the weight and d_i is the depth of leaf i. For given stochastic leaf weights, we are interested in mobiles whose average depth is as small as possible. The following figure shows a stochastic mobile with five leaves and average depth $9/4$.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press



(i) Prove that the average depth of any stochastic mobile with leaf weights $p_1, \ldots, p_n$ is at least $H(p_1, \ldots, p_n)$. Hint: Induction on n.

(ii) For given $p_1, \ldots, p_n \in \mathbb{R}_{>0}$ such that $\sum_{1 \le i \le n} p_i = 1$, let $l_i = -\lfloor \log p_i \rfloor > 0$ for $1 \le i \le n$, $l = \max\{l_i : 1 \le i \le n\}$, and n_j be the number of indices i such that $l_i = j$, for $1 \le j \le l$. Prove that $\sum_{1 \le j \le l} n_j 2^{-j} \le 1$.

(iii) Consider the following algorithm, which uses ideas by Shannon (1948), Fano (1949, 1961), and Kraft (1949) for constructing a stochastic mobile of small average depth.

ALGORITHM 10.26 Building a mobile.
Input: $p_1, \ldots, p_n \in \mathbb{R}_{>0}$ such that $\sum_{1 \le i \le n} p_i = 1$.
Output: A stochastic mobile with leaf weights $p_1, \ldots, p_n$.

1. let $l_1, \ldots, l_n, l, n_1, \ldots, n_l$ be as in (ii), and create a full binary tree t (that is, a complete binary tree with 2^l leaves of depth l) with all node weights equal to zero
2. **for** $j = 1, \ldots, l$ **do**
3. assign those weights p_i with $l_i = j$ to the first n_j of the nodes of depth j in t and remove the subtree of each such node with positive weight from t
4. **for** $j = l, l-1, \ldots, 1$ remove the leaves of depth j with zero weight in t
5. **while** the tree t is not complete, so that there exists a node with a single child, identify that node with its child and remove the edge between them
6. compute the weights of the inner nodes in t, proceeding from the leaves to the root
   return t

Use (ii) to show that before the jth pass through step 3, there are precisely $2^j - n_1 2^{j-1} - n_2 2^{j-2} - \cdots - n_{j-1} \cdot 2 \ge n_j$ nodes of depth j left in t, and conclude that the algorithm works correctly.

(iv) Prove that the average depth of t after step 4 is less than $H(p_1, \ldots, p_n) + 1$, and conclude that this is true as well for the tree returned in step 6.

(v) Run the algorithm with $p_1 = p_3 = p_7 = p_8 = 1/17$, $p_2 = 5/17$, $p_4 = p_5 = 2/17$, and $p_6 = 4/17$.

10.6* This exercise discusses Huffman (1952) codes, a tool for data compression. Suppose that we want to encode a piece of text over a finite alphabet $\Sigma = \{\sigma_1, \ldots, \sigma_n\}$ in binary using as few bits as possible. If we know nothing more than the size of Σ, then there seems to be no better way than to choose some encoding of the elements Σ as bit strings of fixed length $\lceil \log n \rceil$. Suppose now that for each element σ_i, we know the frequency p_i with which it occurs in our text. The idea of the Huffman code is then to use a variable-length encoding which encodes letters that occur frequently by shorter bit strings than letters that only rarely occur. Huffman codes are **instantaneous codes**, so that no codeword is a prefix of another codeword, and can be represented by binary trees.

Here is an algorithm which dynamically builds a stochastic mobile (Exercise 10.5) with leaf weights $p_1, \ldots, p_n$, the **Huffman tree**, which has minimal average depth.

ALGORITHM 10.27 Building a Huffman tree.
Input: $p_1, \ldots, p_n \in \mathbb{R}_{>0}$ such that $p_1 + \cdots + p_n = 1$.
Output: A Huffman tree for $p_1, \ldots, p_n$.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

1. create n nodes $t_1,\dots,t_n$, such that t_i has weight p_i,
   $T \longleftarrow \{t_1,\dots,t_n\}$
2. **repeat**
3. choose two trees $t^*,t^{**} \in T$ with root weights p^*,p^{**}, such that $p^* \le p^{**}$ and the roots of all other trees in T have weight at least p^{**}
4. create a new tree t with root weight p^*+p^{**} and children t^* and t^{**}
5. $T \longleftarrow (T \setminus \{t^*,t^{**}\}) \cup \{t\}$
6. **until** $\#T = 1$
7. **return** the single tree in T

(i) Build a Huffman tree for "MISSISSIPPI_RIVER" over the alphabet $\Sigma = \{\mathrm{E},\mathrm{I},\mathrm{M},\mathrm{P},\mathrm{R},\mathrm{S},\mathrm{V},_\}$. (There is some non-determinism in step 3 if there are several trees with minimal root weights, so the Huffman tree is not unique.)

(ii) To encode a piece of text, the two edges from a node to its two children in the Huffman tree are labeled 0 and 1, and the encoding of σ_i is obtained by concatenating the edge labels on the path from the root to leaf i of weight p_i. The average length of a codeword is then the average depth of the tree. Encode the text from (i), both with the Huffman tree and with the tree from Exercise 10.5, and compare the length of the resulting bit string to the length when using a fixed-length 3 bit code.

It is possible to show that the Huffman tree has minimal average depth (see, for example, Hamming (1986), §4.8, or Knuth (1997), §2.3.4.5), and Exercise 10.5 implies that its average depth is less than $H(p_1,\dots,p_n)+1$.

10.7* Let R be a ring (commutative, with 1), $m_0,\dots,m_{r-1} \in R[x]$ nonconstant, $m = m_0 \cdots m_{r-1}$ of degree n, and $p_i = (\deg m_i)/n$ for all i. Let t be a stochastic mobile for $p_0,\dots,p_{r-1}$ with average depth d, as in Exercise 10.5.

(i) Prove that $d+1 = \sum_v p(v)$, where the sum is over all nodes of t and $p(v)$ is the weight of node v. Conclude that d is the sum of the weights of all internal nodes in t.

(ii) Show that computing m along the tree t, with m_i at the leaf of weight p_i, takes at most $d\,\mathsf{M}(n)$ operations in R. You may assume that $\mathsf{M}(n) = nS(n)$ for a monotonic function S. Conclude that m can be computed with less than $\mathsf{M}(n)(H(p_0,\dots,p_{r-1})+1)$ operations.

(iii) Now assume that $\deg m_0 = 1$ and $\deg m_i = 2^{i-1}$ for $1 \le i < r$. Show that $H(p_0,\dots,p_{r-1}) = 2$ and find a stochastic mobile for $p_0,\dots,p_{r-1}$ of average depth 2. Conclude that you can compute m using at most $2\mathsf{M}(n)$ operations in R, and compare this to the bound from Lemma 10.4. Using Exercise 8.34, reduce the bound to $\mathsf{M}(n)+O(n)$.

10.8 Let $r = 2^k$ for some $k \in \mathbb{N}$, $m_0,\dots,m_{r-1}$ positive integers of length $\lambda(m_i) \le l$ (see Section 2.1), and $M_{ij} \in \mathbb{N}$ for $0 \le i \le k$ and $0 \le j < 2^{k-i}$ as in (1).

(i) Find a close upper bound on the length of $M_{i,j}$ depending only on i and l.

(ii) Prove that all $M_{i,j}$ can be computed using $O(\mathsf{M}(rl)\log r)$ word operations.

10.9 (Montgomery 1992) Let R be a ring (commutative, with 1), $m_0,\dots,m_{r-1} \in R[x]$ monic of degree $d > 0$, the subproducts M_{ij} as in (1) for all i,j, and $n = rd$ for a power $r = 2^k$ of 2.

(i) Use Exercise 9.5 to show that all $\mathrm{rev}(M_{ij})^{-1} \bmod x^{2^i d}$ for $i < k$ can be computed with $(k+1)\cdot(\mathsf{M}(n)+O(n))$ ring operations.

(ii) Conclude from Exercise 10.3 and (i) that the fast multipoint evaluation algorithm 10.7 can be modified so as to use only $(1+\frac{7}{2}\log n)(\mathsf{M}(n)+O(n))$ arithmetic operations in R.

10.10 Trace Algorithm 10.11 on computing the interpolating polynomial $f \in \mathbb{Q}[x]$ of degree less than 4 such that $f(u) = 2^u$ for $u = 0,1,2,3$.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

10.11 In this exercise, you are to examine the cost for several evaluations and interpolations at the same set of points. Show that one evaluation can be done with at most $(2\mathsf{M}(n)+O(n))\log n$ operations in R and one interpolation with at most $(\mathsf{M}(n)+O(n))\log n)$ operations if the cost for precomputing data depending only on the points is ignored. Hint: Some preconditioning on the divisors in the "going down" is possible; see Section 9.1.

10.12* Let $n=2^k$ be a power of 2, F a field, $u_0,\ldots,u_{n-1}\in F$ distinct, and $v_0,\ldots,v_{n-1}\in F$. You are to design an interpolation algorithm with running time $O(\mathsf{M}(n)\log^2 n)$.

(i) Let $m_1,m_2\in F[x]$ of degree n be monic and coprime and $v_1,v_2\in F[x]$ of degree less than n. Using Theorem 11.10, give an algorithm that computes a solution $f\in F[x]$ of degree less than $2n$ of the congruences $f\equiv v_1 \bmod m_1$ and $f\equiv v_2 \bmod m_2$, taking $O(\mathsf{M}(n)\log n)$ operations in F.

(ii) Use (i) to design a divide-and-conquer algorithm computing the interpolating polynomial $f\in F[x]$ of degree less than n such that $f(u_i)=v_i$ for all i, and show that it takes $O(\mathsf{M}(n)\log^2 n)$ operations in F.

(iii) Trace your algorithm on the example from Exercise 10.10.

10.13* Prove Theorem 10.13. Hint: Exercise 6.15.

10.14* Prove the following generalized version of the Chinese Remainder Theorem. Let R be a ring (commutative, with 1), $I_0,\ldots,I_{r-1}$ ideals, and $I=I_0\cap\cdots\cap I_{r-1}$. If $I_i+I_j=R$ for $i\neq j$, then the map

$$\begin{array}{rcl}\chi: R/I & \longrightarrow & R/I_0\times\ldots\times R/I_{r-1},\\ f \bmod I & \longmapsto & (f \bmod I_0,\ldots,f \bmod I_{r-1})\end{array}$$

is an isomorphism.

10.15 Prove that Algorithm 10.14 works correctly.

10.16* Prove Theorem 10.21.

10.17* (i) Prove that when all moduli have the same degree, then the fast modular reduction algorithm 10.16 takes only $(11\mathsf{M}(n/2)+O(n))\log r$ or $(\frac{11}{2}\mathsf{M}(n)+O(n))\log r$ ring operations and Algorithm 10.20 takes only $(\mathsf{M}(n)+O(n))\log r$ operations.

(ii) Show that Algorithm 10.18 takes at most $(11\log r+24\log(n/r)+5)\mathsf{M}(n)+O(n\log n)$ arithmetic operations if R is a field and all moduli have the same degree, using (i) and Theorem 11.10.

(iii) Conclude that the fast Chinese Remainder Algorithm 10.22 takes at most $(24\mathsf{M}(n)+O(n))\cdot\log n$ arithmetic operations if R is a field and all moduli have the same degree.

You may find Exercise 10.3 useful.

10.18 Let F be a field, $f_1,\ldots,f_r\in F[x]$ pairwise coprime, $e_1,\ldots,e_r\in\mathbb{N}_{>0}$, $f=f_1^{e_1}\cdots f_r^{e_r}$, and $g\in F[x]$ of degree less than $\deg f$. Show that the partial fraction decomposition of g/f with respect to the given factorization of f (see Section 5.11) can be computed with $O(\mathsf{M}(n)\log n)$ field operations.

10.19* Work out the details for the integer versions of Algorithms 10.14 and 10.20, and prove Theorems 10.24 and 10.25.

10.20⟶ You are to trace the integer analog of Algorithms 10.14 and 10.20. The point is to see how the algorithm works, not just to compute the final result. Let $m_0=23, m_1=24, m_2=25$, and $m_3=29$.

(i) Check that the moduli are pairwise coprime.

(ii) Compute the binary tree of products.

(iii) Compute $300\,000 \bmod m_i$ for $0\le i<3$, using Algorithm 10.14.

(iv) Let $v_0=5, v_1=3, v_2=1$, and $v_3=22$. Use the fast Chinese Remainder Algorithm to compute $f\in\mathbb{Z}$ such that $f\equiv v_i \bmod m_i$ for $0\le i<4$.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

10.21* Let $a, b \in \mathbb{Z}[x]$ nonzero such that $\deg b < \deg a = n$ and $\|a\|_\infty \leq A$. You are to design a small primes modular algorithm that decides whether $b \mid a$, and if so, computes the quotient $q = a/b \in \mathbb{Z}[x]$. By Mignotte's bound 6.33, we have $\|b\|_1 \|q\|_1 \leq B = (n+1)^{1/2} 2^n A$ in the latter case. The algorithm should choose a collection of distinct primes $p_1, \ldots, p_r < 2r \log r$ not dividing $\mathrm{lc}(b)$, with r chosen appropriately such that their product exceeds $2B$, calculate $(a \bmod p_i)/(b \bmod p_i)$ for all i (if this is not possible, then certainly $b \nmid a$), compute a trial quotient q by Chinese remaindering, and finally check whether $\|b\|_1 \|q\|_1 \leq B$. Work out the details, prove that this procedure works correctly, and show that it takes $O((\mathsf{M}(n) + \log\log\log B) \log B \cdot \mathsf{M}(\log\log B) + n\mathsf{M}(\log B \log\log B) \log\log B)$ or $O^\sim(n^2 + n \log A)$ word operations. You may ignore the cost of $O(\log B (\log\log B)^2 \log\log\log B)$ word operations for finding the small primes (Theorem 18.10). Use Corollary 11.13 for arithmetic in $\mathbb{F}_{p_i}$. See also Exercises 6.26 and 9.14.

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press

The mathematically sophisticated will know how to skip formulæ.
This skill is easy to practice for others also.

Leslie G. Valiant (1994)

Thus it appears that whatever may be the number of digits
the Analytical Engine is capable of holding, if it is required to
make all the computations with k times that number of digits,
then it can be executed by the same Engine,
but in an amount of time equal to k^2 times the former.

Charles Babbage (1864)

Τόδε ἤδη ἐπεσκέψω, ὡς οἵ τε φύσει λογιστικοὶ εἰς πάντα τὰ μαθήματα ὡς ἔπος εἰπεῖν ὀξεῖς φύονται, οἵ τε βραδεῖς, ἂν ἐν τούτῳ παιδευθῶσι καὶ γυμνάσωνται, κἂν μηδὲν ἄλλο ὠφεληθῶσιν, ὅμως εἴς γε τὸ ὀξύτεροι αὐτοὶ αὐτῶν γίγνεσθαι πάντες ἐπιδιδόασιν.[1]

Plato (c. 375 BC)

[1] Have you [Glaukon] ever noticed that those who have a talent for mathematics are, almost without exception, talented in all sciences? And that mentally slow people, if they be trained and exercised in this study, become invariably quicker than they were before, even if they draw no other profit from it?

https://doi.org/10.1017/CBO9781139856065.013 Published online by Cambridge University Press